Task5:

String to Integer (atoi)

Aim:-To determine the Convert a string into a 32-bit signed integer (int in Java).

Algorithm:-

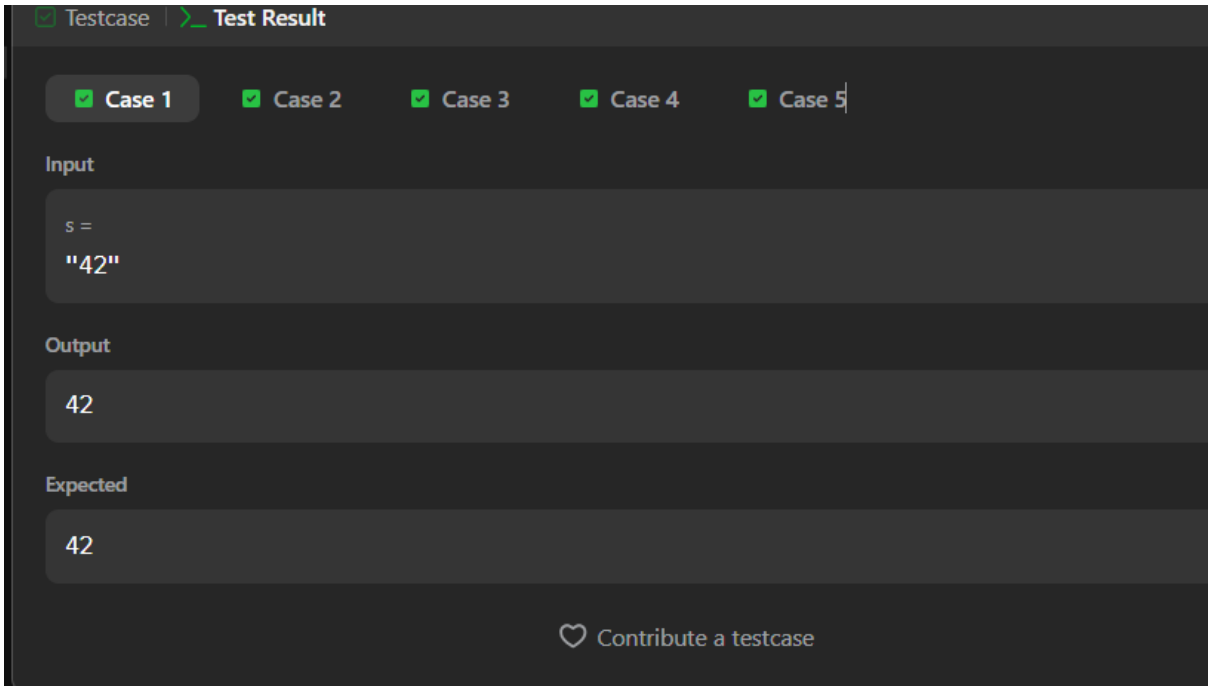
1. **Skip whitespace** at the start.
2. **Check sign** (+ or -).
3. **Parse digits** one by one until a non-digit.
4. **Handle overflow** by clamping to Integer.MAX_VALUE or Integer.MIN_VALUE.
5. **Return result** with sign applied.

Program:-

```
class Solution {
   public int myAtoi(String s) {
      int i = 0, n = s.length();
      while (i < n && s.charAt(i) == ' ') i++;
      if (i == n) return 0;

      int sign = 1;
      if (s.charAt(i) == '+' || s.charAt(i) == '-') {
         sign = (s.charAt(i) == '-') ? -1 : 1;
         i++;
      }

      long num = 0;
      while (i < n && Character.isDigit(s.charAt(i))) {
         num = num * 10 + (s.charAt(i) - '0');
```

```
            if (sign == 1 && num > Integer.MAX_VALUE) return
Integer.MAX_VALUE;
            if (sign == -1 && -num < Integer.MIN_VALUE) return
Integer.MIN_VALUE;
            i++;
        }

        return (int)(sign * num);
    }
}
```

Output:-



Result:-Thus,to determine the Convert a string into a 32-bit signed integer (int in Java) is are successfully executed.

Task6:

Alternating Characters

Aim:-To Transform a string of only 'A' and 'B' characters so that no two adjacent characters are the same.

Algorithm:-

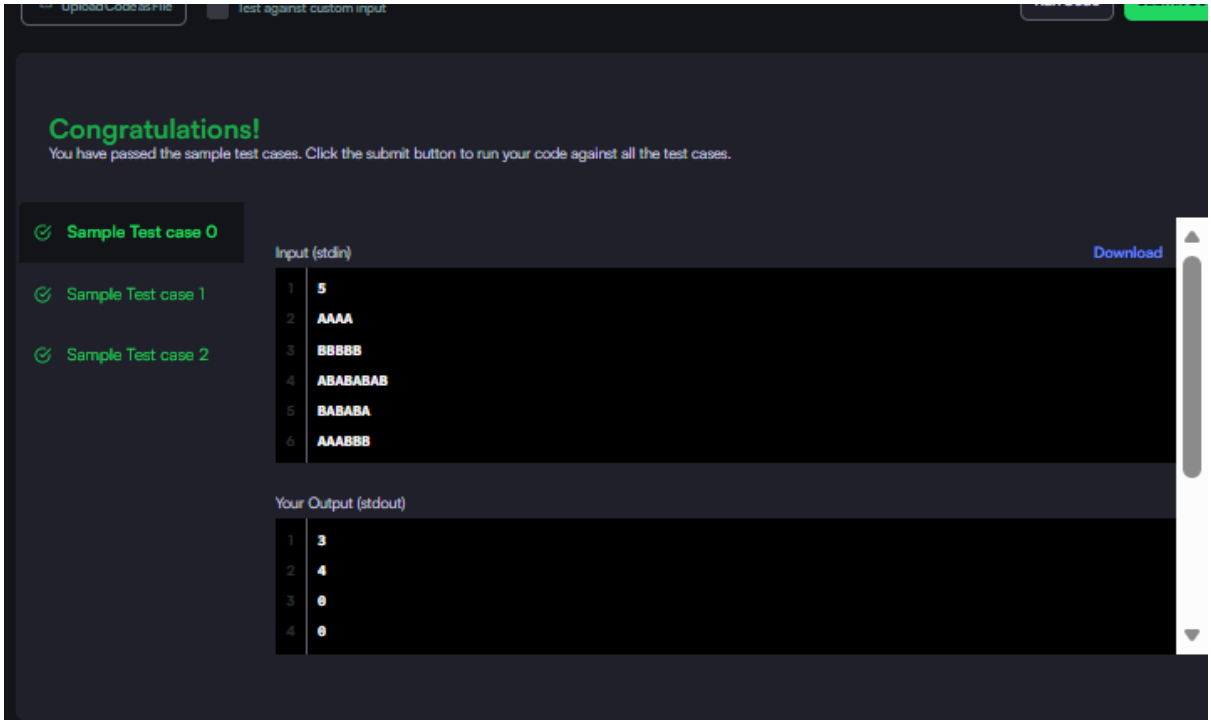
1. Initialize a counter deletions = 0.
2. Traverse the string from left to right.
3. For each character, compare it with the previous one:
    - If they are the same, increment deletions.
    - Otherwise, continue.
4. Return deletions as the result.

Program:-

```
public class Solution {
   static int alternatingCharacters(String s) {
      int deletions = 0;
      for (int i = 1; i < s.length(); i++) {
         if (s.charAt(i) == s.charAt(i - 1)) {
            deletions++;
         }
      }
      return deletions;
   }

   public static void main(String[] args) {
      java.util.Scanner sc = new java.util.Scanner(System.in);
```

```
        int q = sc.nextInt();
        while (q-- > 0) {
            String s = sc.next();
            System.out.println(alternatingCharacters(s));
        }
        sc.close();
    }
}
```

Output:-



Result:-Thus,To Transform a string of only 'A' and 'B' characters so that no two adjacent characters are the same is are successfully executed.

Task7:

Longest Substring Without Repeating Characters

Aim:-To determine Find the length of the longest substring of a given string s that contains no duplicate characters.

Algorithm:-

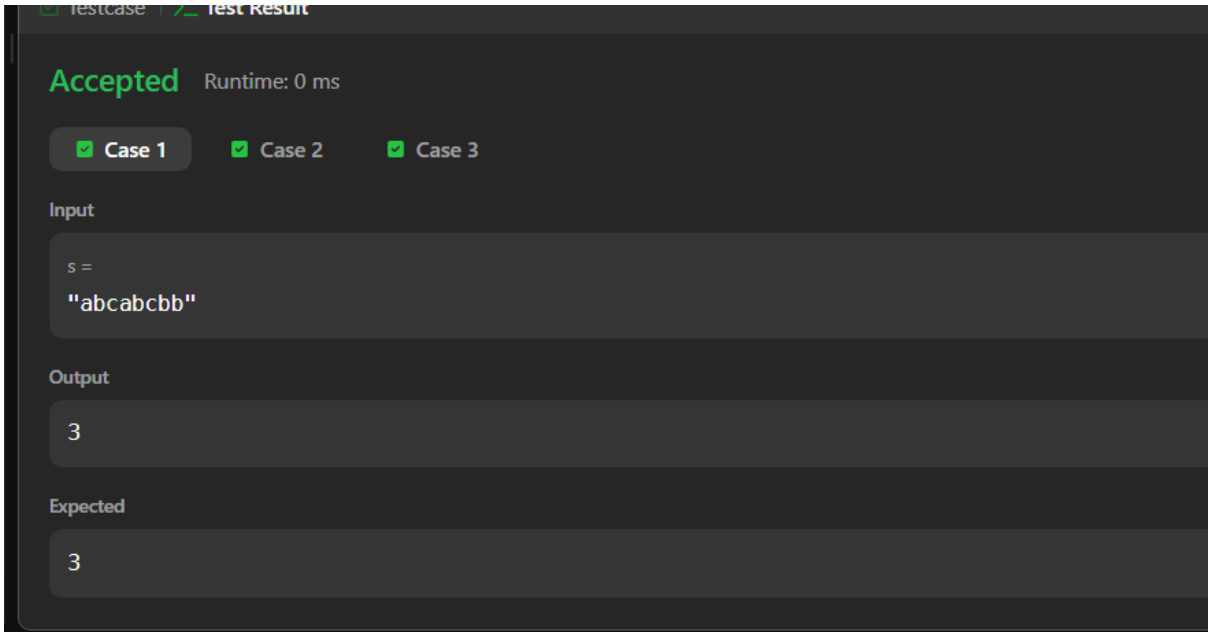
1. Use two pointers (left and right) to represent the current window.
2. Maintain a set or map to track characters in the current window.
3. Expand right pointer one step at a time:
4. If the character is not in the set, add it and update max length.
5. If the character is already in the set, move left forward until the duplicate is removed.
6. Continue until the end of the string.
7. Return the maximum length found.

Program:-

```
import java.util.*;

class Solution {

   public int lengthOfLongestSubstring(String s) {

      Set<Character> set = new HashSet<>();

      int left = 0, maxLen = 0;

      for (int right = 0; right < s.length(); right++) {

         while (set.contains(s.charAt(right))) {

            set.remove(s.charAt(left));

            left++;

         }

         set.add(s.charAt(right));

         maxLen = Math.max(maxLen, right - left + 1);

      }
```

```
        return maxLen;
    }
}
```

Output:-



Result:-Thus,To determine Find the length of the longest substring of a given string s that contains no duplicate characters is are successfully executed.

Task8:

Find and Replace Pattern

Aim:-To determine a list of strings words and a string pattern, return a list of words[i] that match pattern. You may return the answer in any order.

Algorithm:-

1. For each word in words:
    - Check if it can be mapped to the pattern.
    - Use two hash maps:
        - pToW → maps pattern characters to word characters.
        - wToP → maps word characters to pattern characters.
    - Traverse both strings simultaneously:
        - If a mapping exists, verify consistency.
        - If no mapping exists, create one.
    - If all characters match consistently, add the word to the result.
2. Return the list of matching words.

Program:-

```
import java.util.*;
class Solution {
   public List<String> findAndReplacePattern(String[] words, String pattern) {
      List<String> result = new ArrayList<>();
      for (String word : words) {
         if (matches(word, pattern)) {
            result.add(word);
         }
      }
      return result;
```

```
    }
    private boolean matches(String word, String pattern) {
        Map<Character, Character> pToW = new HashMap<>();
        Map<Character, Character> wToP = new HashMap<>();
        for (int i = 0; i < word.length(); i++) {
            char w = word.charAt(i);
            char p = pattern.charAt(i);
            if (pToW.containsKey(p) && pToW.get(p) != w) return false;
            if (wToP.containsKey(w) && wToP.get(w) != p) return false;
            pToW.put(p, w);
            wToP.put(w, p);
        }
        return true;
    }
}
```

Output:-

Result:-Thus,To determine a list of strings words and a string pattern, return *a list of* words[i] *that match* pattern. You may return the answer in any order is are successfully executed.

Task9:

String Matching in an Array

Aim:- Given an array of strings words, return all strings that are substrings of another word in the array.

Algorithm:-

1. Iterate through each word in words.
2. For each word, check if it is contained in any other word in the list.
3. If yes, add it to the result list.
4. Return the result list. This is straightforward since constraints are small (words.length <= 100, word.length <= 30).

Program:-

```
import java.util.*;
class Solution {
   public List<String> stringMatching(String[] words) {
      List<String> result = new ArrayList<>();
      for (int i = 0; i < words.length; i++) {
         for (int j = 0; j < words.length; j++) {
            if (i != j && words[j].contains(words[i])) {
               result.add(words[i]);
               break;
            }
         }
      } return result;
   }
}
```

Output:-

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

words =

```
["mass","as","hero","superhero"]
```

Output

```
["as","hero"]
```

Expected

```
["as","hero"]
```

Result:-Thus,To Given an array of strings words, return all strings that are substrings of another word I n the array is are successfully executed.